

## 7.1 QU'EST-CE QU'UNE ONTOLOGIE ?

Une ontologie est une carte conceptuelle d'un domaine.

Elle répond à trois questions fondamentales :

- (1) Quelles entités existent dans ce domaine ?
- (2) Quelles propriétés ont ces entités ?
- (3) Quelles relations existent entre elles ?

En informatique, une ontologie devient un schéma de base de données.

Dans une capsule Gykhamine, l'ontologie de votre contexte local détermine entièrement la structure de vos modèles (models.py).

ERREUR COMMUNE : Commencer à coder avant d'avoir modélisé.

RÈGLE D'OR : Dessiner l'ontologie SUR PAPIER avant d'écrire une seule ligne de models.py.

## 7.2 TROIS TYPES DE RELATIONS FONDAMENTALES

| RELATION                    | SIGNIFICATION & CODE DJANGO                                                                                  |
|-----------------------------|--------------------------------------------------------------------------------------------------------------|
| EST-UN (héritage)           | Un Médecin EST UN Employé<br>class Medecin(Employe): ...                                                     |
| A-UN (composition)<br>1 → N | Une Commande A UN Client<br>client = ForeignKey(Client, ...)                                                 |
| FAIT PARTIE DE<br>N ↔ M     | Un Produit FAIT PARTIE D'UNE Commande<br>commandes = ManyToManyField(Commande,<br>through='LigneDeCommande') |

## 7.3 EXEMPLE – ONTOLOGIE D'UNE GESTION HOSPITALIÈRE

### ÉTAPE 1 – IDENTIFIER LES ENTITÉS :

Personne, Patient, Médecin, Infirmier, Consultation, Diagnostic, Médicament, Prescription, Chambre, Service, Paiement

### ÉTAPE 2 – CLASSER LES RELATIONS :

Patient EST UNE Personne  
Médecin EST UN Personne  
Infirmier EST UN Personne

Consultation A UN Patient  
Consultation A UN Médecin  
Consultation A UN Diagnostic

Prescription FAIT PARTIE DE Consultation  
Médicament FAIT PARTIE DE Prescription (M:N)

Patient EST DANS Chambre (pendant une hospitalisation)  
Chambre FAIT PARTIE DE Service

### ÉTAPE 3 – TRADUIRE EN MODELS.PY :

```
class Personne(models.Model):
    nom = models.CharField(max_length=100)
    prenom = models.CharField(max_length=100)
    telephone = models.CharField(max_length=20, blank=True)
    class Meta: abstract = True # Classe de base, pas de table propre

class Patient(Personne):
    numero_dossier = models.CharField(max_length=20, unique=True)
```

```

groupe_sanguin = models.CharField(max_length=5, blank=True)
date_naissance = models.DateField(null=True, blank=True)

class Medecin(Personne):
    specialite = models.CharField(max_length=100)
    numero_ordre = models.CharField(max_length=30)

class Service(models.Model):
    nom = models.CharField(max_length=100)
    chef_service = models.ForeignKey(Medecin, null=True, on_delete=models.SET_NULL)

class Chambre(models.Model):
    numero = models.CharField(max_length=10)
    service = models.ForeignKey(Service, on_delete=models.CASCADE)
    capacite = models.IntegerField(default=1)

class Consultation(models.Model):
    patient = models.ForeignKey(Patient, on_delete=models.PROTECT)
    medecin = models.ForeignKey(Medecin, on_delete=models.PROTECT)
    date = models.DateTimeField(auto_now_add=True)
    motif = models.TextField()
    diagnostic = models.TextField(blank=True)

class Medicament(models.Model):
    nom = models.CharField(max_length=200)
    dosage_std = models.CharField(max_length=50)

class Prescription(models.Model):
    consultation = models.ForeignKey(Consultation, on_delete=models.CASCADE)
    medicament = models.ForeignKey(Medicament, on_delete=models.PROTECT)
    posologie = models.CharField(max_length=200)
    duree_jours = models.IntegerField()

```

---

#### 7.4 ONTOLOGIE FORMELLE AVEC NETWORKX (DANS UNE CAPSULE)

---

Pour visualiser et naviguer dans votre ontologie depuis la capsule :

```

import networkx as nx
import json

def construire_ontologie_hospitaliere():
    G = nx.DiGraph()

    # Nœuds (entités)
    entites = ['Personne', 'Patient', 'Médecin', 'Consultation',
               'Médicament', 'Prescription', 'Chambre', 'Service']
    for e in entites:
        G.add_node(e)

    # Arêtes (relations)
    relations = [
        ('Patient', 'Personne', 'EST_UN'),
        ('Médecin', 'Personne', 'EST_UN'),
        ('Consultation', 'Patient', 'IMPLIQUE'),
        ('Consultation', 'Médecin', 'RÉALISÉE_PAR'),
        ('Prescription', 'Consultation', 'RÉSULTE_DE'),
        ('Prescription', 'Médicament', 'CONTIENT'),
        ('Patient', 'Chambre', 'OCCUPE'),
        ('Chambre', 'Service', 'APPARTIENT_À'),
    ]
    for src, dst, rel in relations:
        G.add_edge(src, dst, relation=rel)

    return G

def ontologie_vers_json(G):
    """Convertit le graphe en JSON pour visualisation vis.js"""
    nodes = [{ 'id': n, 'label': n } for n in G.nodes()]
    edges = [{ 'from': u, 'to': v, 'label': d['relation'] }
              for u, v, d in G.edges(data=True)]
    return json.dumps({'nodes': nodes, 'edges': edges})

```

---

```
[Domaine réel]
  ↓ Observer, interviewer les utilisateurs
[Ontologie sur papier]
  ↓ Traduire en classes Python
[models.py dans la capsule]
  ↓ makemigrations + migrate
[Base de données SQLite]
  ↓ Saisie des données par les utilisateurs
[Données réelles]
  ↓ Django ORM + pandas
[Analyse et rapports]
  ↓ Visualiser, améliorer
[Retour au domaine réel]
```

Ce cycle est universel. Il s'applique à tout contexte africain :

- Gestion d'une coopérative agricole à Sibiti
- Suivi de chantiers à Pointe-Noire
- ERP d'une brasserie artisanale à Dolisie
- Registre foncier à Owando

---

### RÉSUMÉ DU CHAPITRE 7

---

- ✓ Ontologie = carte conceptuelle du domaine (entités + relations)
- ✓ 3 relations : EST-UN (héritage), A-UN (ForeignKey), FAIT PARTIE DE (M2M)
- ✓ Modéliser SUR PAPIER avant de coder
- ✓ NetworkX permet de visualiser l'ontologie comme un graphe
- ✓ Ontologie → models.py → base de données → analyse : cycle universel

→ SUITE : Chapitre 8 – Adapter une capsule existante à un nouveau domaine